

Break Amdahl's Law!

Here's a law you should break early and often

By Herb Sutter

Herb is a software architect at Microsoft and chair of the ISO C++ Standards committee. He can be contacted at www.gotw.ca.

Back in 1967, Gene Amdahl famously pointed out what seemed like a fundamental limit to how fast you can make your concurrent code: Some amount of a program's processing is fully " $O(N)$ " parallelizable (call this portion p), and only that portion can scale directly on machines having more and more processor cores. The rest of the program's work is " $O(1)$ " sequential (s). [1,2] Assuming perfect use of all available cores and no parallelization overhead, Amdahl's Law says that the best possible speedup of that program workload on a machine with N cores is given by

$$\text{Speedup} = \frac{s + p}{s + p/N}$$

Note that, as N increases to infinity, the best speedup we can ever get is $(s+p)/s$. Figure 1 illustrates why a program that is half scalably parallelizable and half not won't scale beyond a factor of two even given infinite cores. Some people find this depressing. They conclude that Amdahl's Law means there's no point in trying to write multicore- and manycore-exploiting applications except for a few "embarrassingly parallel" patterns and problem domains with essentially no sequential work at all.

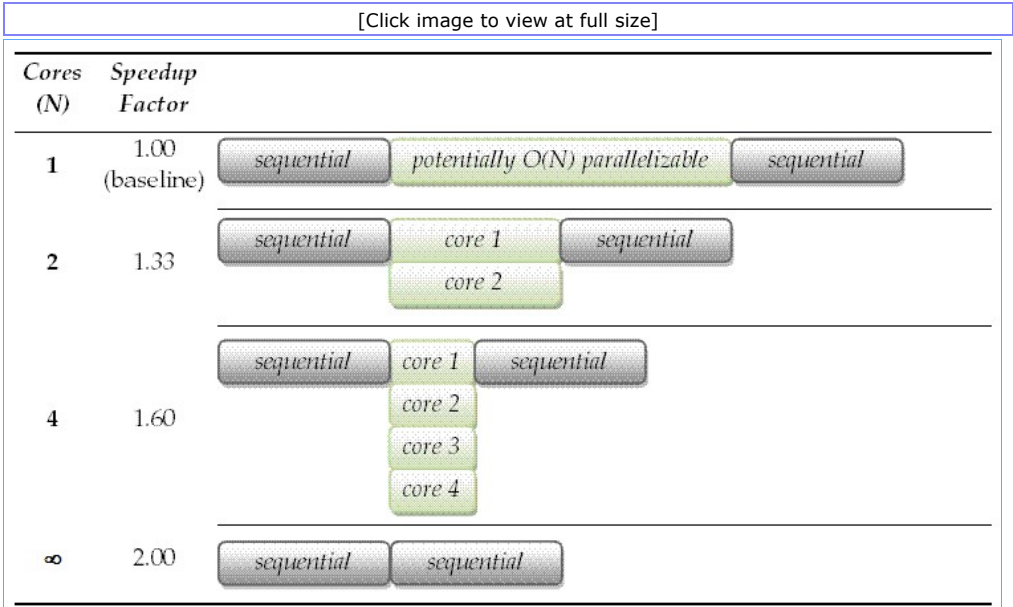


Figure 1: Illustrating Amdahl's Law for $s = p$.

Fortunately, they're wrong. If Amdahl's Game is rigged, well then, to paraphrase a line from the movie *WarGames*: The only way to win is not to play.

Breaking the Speed Limit

Even if your current application doesn't have much embarrassingly parallel functionality, the key observation is that s and p are not fixed. You can change them, including by applying not only $O(N)$, but also $O(K)$ concurrency:

1. You can increase p by doing more of the same: Increase the volume of data processed by the parts that are $O(N)$ parallelizable (and therefore the percentage p of time spent in computing). This is Gustafson's Law.
2. You can increase p by doing more of something new: Add new features that are $O(N)$ parallelizable.
3. You can reduce s by partially parallelizing it using $O(K)$ techniques, such as pipelining.

Let's consider each of these.

1. Do More of the Same $O(N)$ Work

Amdahl analyzed the effect on execution time assuming that we add more processors or cores while keeping the workload fixed. In 1988, John Gustafson came along and said (in effect): But that's not what really happens! When we get more horsepower, we solve bigger problems. "Hence," Gustafson suggested, "it may be most realistic to assume that *run time, not problem size*, is constant." [3]

I'm sure that resonates with your own experience: When it comes to performance criteria that our software must meet, what constraint tends to be the most immovably fixed? It's how much time we're allowed to use to get a useful result. Users have finite patience. For example, they want to wait:

- Less than a second for a response to a mouse click
- Less than a minute to sync a PDA or do an incremental compile
- Less than an hour to encode and burn a DVD
- Less than 12 hours to calculate tomorrow's weather forecast or do a nightly build

and they push back hard if we don't meet those deadlines. But if our users can compile or sync or burn N times as much in the same time given N cores, they will happily accept (nay, greedily gobble) that capability and buy the hardware that matches their problem size.

If we keep run time constant and focus instead on increasing the problem size, we get a much better formula. Again assuming no overhead, Gustafson's Law says that the total work a program can perform in a fixed time on a machine with N cores is given by:

$$\text{Total Work} = s + Np$$

where "Total Work" is equivalent to the "best possible speedup" compared to hypothetically doing all that work sequentially, even if nobody would ever wait for such a sequential execution. For example, Figure 2 illustrates applying Gustafson's Law to the problem in Figure 1. That wonderful amplification of $O(N)$ code just makes our parallel day. Importantly, note that the total program is now $O(N)$, even though there is a sequential portion. As N increases to infinity, the total work we can accomplish also increases to infinity. In practice, this means that it increases until we can't make the problem any bigger or are bound on some other factor, such as memory or other I/O. Still, the key is that the concurrency is scalable, and is not hardwired to a predetermined preferred fixed number K of cores, or worse still bound by a constant factor maximum speedup over the original code as in Amdahl's approach.

[Click image to view at full size]

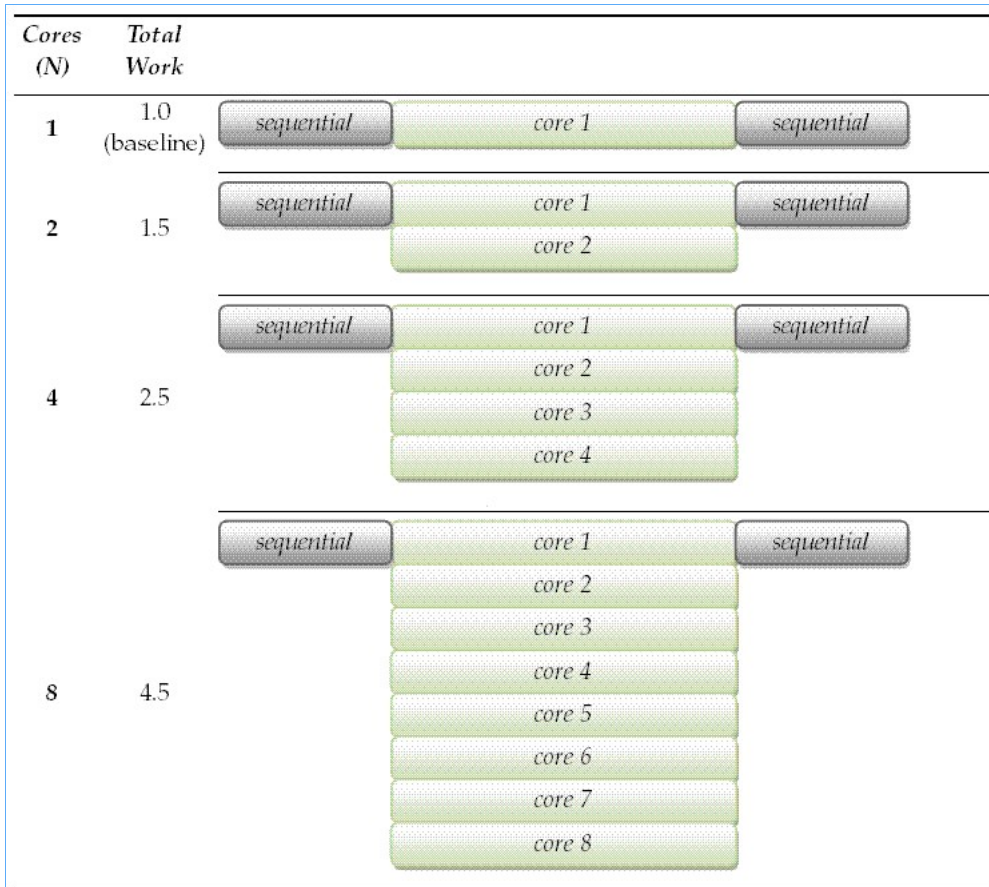


Figure 2: Illustrating Gustafson's Law, total work = $s + Np$.

2. Invent New $O(N)$ Work

Besides solving bigger versions of the same problem, we also have the option of adding new $O(N)$ features. This is especially effective, and it's market changing when the feature is a harder challenge than we could ever solve practically before.

Remember *Myst*? Released in 1993, that first-person immersive adventure was the bestselling computer game of all time for most of the rest of the decade because of its breakthrough graphics. *Myst* let you walk through high-resolution pre-rendered scenes that were like static postcards, preselected views frozen in space and time. Those scenes took hours and hours to render, of course, so all the rendering work had to be done offline and the game was shipped with only the final images, as many as could fit on a CD-ROM. It was simply impossible to think about actually rendering on the user's computer at all, never mind at several frames per second in real time so that the player could actually move around the world and look wherever he wanted. Sure, *Doom* accomplished that later the same year, but not nearly at *Myst*'s high graphic quality.

But real-time 3D was only a matter of time. *Myst* sequels themselves first let the player turn and look at a 360-degree panoramic rendering from a fixed viewpoint, and then completely freed us to walk and look anywhere in real-time 3D.

Just as *Myst* itself was once unthinkable but was made possible by the widespread availability of PCs that had good graphics and CD drives, and real-tin

3D sequels were made possible by advances in both CPUs and parallel graphics hardware, so new features that are unthinkable today will be possible on 16-core and 64-core machines. In each case, it's a matter of finding what previously unthinkable work could now be enabled by new machines, and enabling it—at first by gurus building their own libraries and frameworks, then by more and more developers as the tools became more mainstream.

But don't show me ray-traced bouncing balls or Mandelbrot graphics or the other usual embarrassingly parallel but niche (or downright useless) clichés—what we're looking for are real ideas of real software we could imagine real kids and grandmothers using that could become possible on manycore machines. Here's a quick potential example: Researchers know how to do speech recognition with near-human-quality accuracy in the lab, which is astonishingly good and would enable breakthrough user interfaces if it could be done that reliably in real time. The only trouble is that the software takes a week to run...on a single core. Can it be parallelized, and if so how many cores would we need to get a useful answer in a useful time? Bunches of smart people (and the smart money behind them) are investing hard work not only to find out the answer to that question, but also to find more questions like it.

3. Reduce s Using $O(K)$ Concurrency

Just because something can't be made $O(N)$ parallel doesn't mean it doesn't contain any potential concurrency at all. For the past 30 years, processors have been successfully speeding up purely sequential programs by injecting $O(K)$ concurrency in the form of caching, prefetching, instruction reordering, and pipelining.

Granted, the maximum potential speedup of sequential code is bounded by some fixed constant K . For example, adding a fast Level 1 (L1) cache can improve memory access from slower L2 or L3 cache times (or, worse still, glacial DRAM or geological disk) up to the speed of L1, but no further. Adding a five-stage instruction pipeline can improve performance by a factor of five, assuming all the pipeline stages have equal weight and minimal overhead, but no further. But $O(K)$ improvements are nothing to sneeze at. Today, nobody would buy a desktop PC that didn't have plenty of cache and a pipelined processor.

We can use the same tricks. We can compute separable pieces of work concurrently by running some of the work on a thread pool. For example, given the following sequential function that computes total sales in two independent HASH(0x87e450) of similar size

```
Quantity ComputeSales() { return region1.Sales() // these calculations
                        + region2.Sales();       // are independent
}
```

you can improve this from $O(1)$ to $O(2)$ by moving the computation of *region1.Sales()* to a background thread or thread pool, so that *region1.Sales()* and *region2.Sales()* can execute concurrently. Just be sure that the calculation you're shipping to another thread is big enough to justify today's overhead of executing it somewhere else, and that the data the two operations use really are independent.

Generalizing that technique, we can pipeline to overlap separable subparts of work being done to each piece of data in a collection [1]. Figure 3 illustrates the effect of additionally applying $O(K)$ techniques to the "sequential" portions of the program.

[Click image to view at full size]

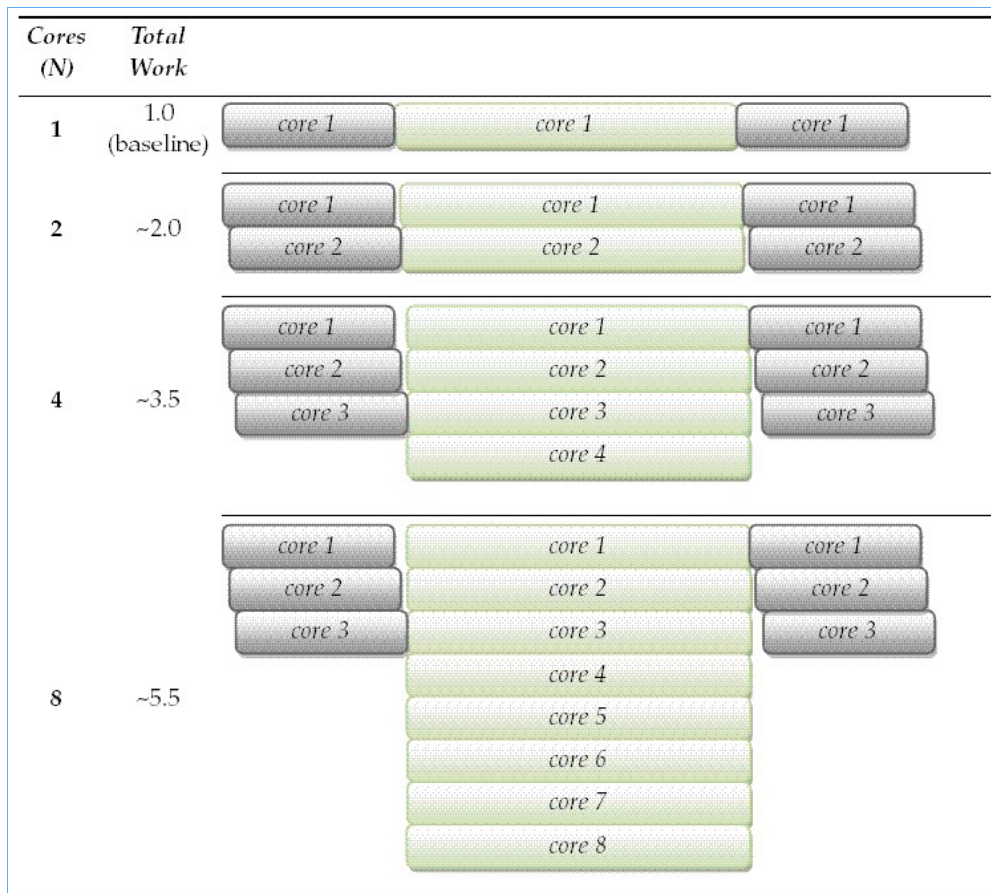


Figure 3: Applying Gustafson's Law to p while also pipelining s .

Conclusion

Over the next few articles, I'll discuss ways to apply $O(N)$ and $O(K)$ techniques, and ways to not only cheat Amdahl, but beat him. Shameless teaser: How can you use N cores to get an answer more than N times faster? Normally, getting just N -fold speedups is considered the Holy Grail. Hint: Think about ways you might leverage data locality and/or perform speculative and cancellable execution to set yourself up for such superlinear speedups of p . Then read next month's column.

Cheat Amdahl by doing more of the same: Solve bigger cases of the same problem. As Gustafson concluded: "Speedup should be measured by scaling the problem to the number of processors, not fixing problem size. We expect to extend our success to a broader range of applications and even larger values for N ." (To put "even larger" in context, when Gustafson wrote that he was already reporting repeatable successes with $N = 1024$ processors.)

Cheat Amdahl by finding new $O(N)$ work to do that was once unthinkable. Cheat him by $O(K)$ -parallelizing the apparently "sequential" parts of your code.

To paraphrase Churchill: We shall cheat him in the data sizes, we shall cheat him in the loops, we shall cheat him in the pipelines and in the caches, we shall cheat him in the unimagined new features. We shall never surrender.

Notes

[1] For more on $O(1)$, $O(K)$, and $O(N)$ concurrency, see my column "How Much Scalability Do You Have or Need?" (*DDJ*, September 2007), available online at <http://www.ddj.com/hpc-high-performance-computing/201202924>. Briefly, $O(x)$ means that the code has about x things it could be running concurrently at a given time, where $O(1)$ is sequential, $O(K)$ is code that hardwires a predetermined fixed amount of concurrency that prefers K cores and can't use $>K$ cores, and $O(N)$ is scalable across variable numbers of cores.

[2] Gene Amdahl. "Validity of the Single Processor Approach to Achieving Large-Scale Computing Capabilities" (*AFIPS Conference Proceedings*, Vol. 30, AFIPS Press, April 1967).

[3] John Gustafson. "Reevaluating Amdahl's Law" (*Communications of the ACM*, 31(5), 1988).